

AWS Developer Exam: Domain 3 – Deploy Code

Using AWS CI/CD Services

Overview

This task statement dives into the specific AWS services used for CI/CD and how they integrate with the broader deployment lifecycle. Key themes include:

- CI/CD fundamentals and goals
- AWS CI/CD tool ecosystem
- CodeDeploy deployment configurations for EC2, ECS, and Lambda
- CloudFormation StackSets for multi-account management
- SAM for serverless deployments
- Stage variables, parameter labels, and certificate/credential rotation

Exam Tip: CI/CD in AWS falls under the **Operational Excellence** pillar of the AWS Well-Architected Framework.

CI/CD Fundamentals

CI/CD is a software development process where code is pushed to a central repository (e.g., GitHub, CodeCommit) and every code push triggers an automated build followed by automated tests.

Main Goals of CI/CD

- Identify issues at **early stages** of development
- Improve **code quality** continuously
- Reduce **validation time**
- Enable frequent and reliable **release updates**

How a CI/CD Pipeline Works

Each stage of the pipeline is a **logical unit** that validates the code as it progresses. Code is improved and continuously verified through each stage before reaching production.

Infrastructure as Code (IaC) and CI/CD

IaC is key to automating the process and lifecycle management of an application in its environment. Every step — provisioning, configuration, deployment — can be programmed and managed as **source code**, making pipelines fully repeatable and auditable.

AWS CI/CD Tool Ecosystem

Know the complete set of AWS CI/CD tools and how they integrate with other services:

Service	Purpose
AWS CodePipeline	Orchestrates the full CI/CD pipeline; automates release workflows across environments and accounts
AWS CodeBuild	Managed build service; compiles code, runs tests, produces deployment artifacts
AWS CodeDeploy	Automates application deployments to EC2, on-premises servers, Lambda, and ECS
AWS CodeArtifact	Managed artifact repository for storing and sharing versioned packages
Amazon ECR	Container image registry; stores and manages Docker images
AWS Copilot	Deploys and operates containerized applications on ECS from source code
AWS CDK	Infrastructure as code using familiar programming languages
AWS SAM	Serverless-focused extension of CloudFormation for building and deploying serverless apps
AWS Amplify	Full-stack development platform for web and mobile apps with built-in CI/CD
AWS X-Ray	Distributed tracing for analyzing and debugging applications in production
AWS CloudShell	Browser-based shell for running AWS CLI commands without local setup

Also know how these tools integrate with: **SAM, Lambda, S3, CloudFormation, AWS AppConfig, and Secrets Manager.**

Deployment Strategies (General)

Know the standard deployment strategies and when to apply each:

Strategy	Description	Downtime Risk
All-at-Once	Deploy to all instances simultaneously	High — all instances affected at once
Rolling	Deploy to a subset of instances at a time	Medium — reduced capacity during rollout
Rolling with Additional Batch	Launch extra instances during rollout to maintain full capacity	Low — capacity maintained throughout
Immutable	Launch a fresh set of instances with the new version; swap when healthy	Very Low — old instances remain until new ones are validated
Blue/Green	Run two environments; shift traffic when new version is validated	Minimal — instant rollback available
Canary	Shift a small percentage of traffic first, then the rest	Low — gradual validation
Linear	Shift traffic in equal increments at equal time intervals	Low — steady, time-based rollout

Exam Scenario: Single Docker container on Elastic Beanstalk — no downtime, no degradation, full compute capacity maintained throughout deployment → **Rolling with Additional Batch + Immutable** together.

AWS CodeDeploy — Deep Dive

Supported Compute Targets

EC2 instances
 On-premises servers
 AWS Lambda functions
 Amazon ECS

Deployment Types by Compute Target

EC2 and On-Premises

In-place — The existing instances are stopped, updated, and restarted.

Blue/Green — New instances are launched with the new version; traffic is shifted after validation.

ECS (via CloudFormation Blue/Green)

Traffic can be shifted using:

Canary — Small percentage shifted first, remainder after validation

Linear — Equal increments at equal intervals

All-at-Once — Entire traffic shifted immediately

Lambda

Canary — Two-increment traffic shift

Linear — Equal increments at equal time intervals

All-at-Once — Immediate full traffic shift

Immutable — Not applicable for Lambda

Know the **predefined deployment configurations** for each compute type (EC2, ECS, Lambda) — CodeDeploy has built-in presets for each.

API Gateway Deployments and Stages

Deploying API Gateway Across Stages

As API versions change, you continuously deploy your API to different stages (dev, test, staging, prod). Understand how to:

Create and manage **API Gateway deployments** and associate them with stages

Use **canary releases** in API Gateway to gradually shift traffic to a new deployment within a stage

Use **existing runtime configurations** to create dynamic deployments

Stage Variables

Stage variables are key-value pairs that act as **environment-specific configuration parameters** in API Gateway. They can be passed to Lambda functions via mapping templates.

Exam Scenario: You need to reuse the same Lambda function across multiple API stages, but each stage should read from a different DynamoDB table. → Use **mapping templates** and a **stage variable** to pass the DynamoDB table name to Lambda dynamically. The Lambda function reads the table name from the stage variable at runtime.

CloudFormation StackSets — Multi-Account Management

When managing multiple AWS accounts (staging, testing, production), updating a CloudFormation template across all accounts individually is tedious.

Solution: CloudFormation StackSets

StackSets allow you to create, update, or delete stacks across **multiple AWS accounts and Regions** with a single CloudFormation operation — with the **least administrative effort**.

Exam Scenario: Several AWS accounts for staging, testing, and production need updates to a shared CloudFormation template → Use **CloudFormation StackSets**.

SAM for Serverless Deployments

When setting up a serverless architecture with Lambda, API Gateway, and DynamoDB as a single stack — and needing to **locally build, test, debug, and deploy** — use **AWS SAM**.

Why SAM Over CloudFormation or Elastic Beanstalk?

SAM is an **extension of CloudFormation**, inheriting all its reliable deployment capabilities.

SAM allows you to define resources using CloudFormation syntax **within the SAM template**.

SAM includes purpose-built **AWS tools for building, testing, and deploying serverless applications** locally and in AWS.

Elastic Beanstalk is designed for traditional application servers, not serverless architectures.

Systems Manager — Application Management Features

AWS Systems Manager includes several capabilities relevant to deployment management:

Feature	Purpose
Parameter Store — Parameter Labels	Assign labels to specific parameter versions for easy reference (e.g., <code>prod</code> , <code>beta</code>)
Parameter Store — Parameter Versions	Track version history of parameters; roll back to previous versions
Application Manager	Troubleshoot and manage applications and their AWS resources
AWS AppConfig	Create, manage, and deploy application configurations with built-in validation and monitoring

Automated Rotation — What Rotates and What Doesn't

Service	Automates Rotation?	What It Rotates
AWS Certificate Manager (ACM)	✔ Yes	SSL/TLS certificates
AWS Secrets Manager	✔ Yes	Secrets, credentials, API keys

Service	Automates Rotation?	What It Rotates
AWS KMS	✔ Yes (for enabled CMKs)	Encryption key material
SSM Parameter Store	✘ No	Does not auto-rotate
IAM Database Authentication	✘ No	Does not auto-rotate credentials

Exam Tip: If a question asks which services automate credential or certificate rotation — **ACM, Secrets Manager, and KMS** do. **Parameter Store and IAM database authentication do not.**

Key Exam Tips

CI/CD falls under the Operational Excellence pillar of the AWS Well-Architected Framework.

CodeDeploy supports in-place and blue/green for EC2/on-premises; ECS supports canary, linear, and all-at-once; Lambda supports canary, linear, all-at-once (not immutable).

Rolling with Additional Batch + Immutable = no downtime and full compute capacity maintained — use for Elastic Beanstalk single-container Docker workloads.

CloudFormation StackSets = manage and update stacks across multiple accounts and Regions with one operation.

SAM is the right choice for serverless architectures needing local build, test, debug, and deploy capabilities — not Elastic Beanstalk or plain CloudFormation.

Stage variables + mapping templates = pass dynamic configuration (e.g., DynamoDB table names) to Lambda functions per API stage without changing code.

Use **canary releases in API Gateway** to gradually shift traffic to a new deployment within a stage.

ACM and Secrets Manager automate rotation — Parameter Store and IAM database authentication do not.

Know the **predefined CodeDeploy deployment configurations** for EC2, ECS, and Lambda — each compute target has its own set of presets.

AWS X-Ray is the tracing and debugging tool in the CI/CD ecosystem — use it to analyze and debug deployed applications. EOF